

# DataHarvest

## Rapport technique

*Framework de scraping modulaire — Projet final*

Tahina HONI-RIKA

Houda OUADAH

Mastère Développement, Data & IA — 5ème année

IPSSI Montpellier

[Date de soutenance]

## 1. Introduction et périmètre

### ***Quel problème DataHarvest résout-il ?***

Les trois premiers jours de ce module ont montré trois façons différentes de scraper un site : des scripts Requests + BeautifulSoup ad hoc, des scripts Selenium pour le JavaScript, et des spiders Scrapy pour l'industrialisation. Dans chaque cas, la logique de récupération HTTP, la logique d'extraction, la logique de validation et la logique de stockage étaient mélangées dans un seul fichier. Modifier un site signifiait généralement réécrire tout le script.

DataHarvest répond à un problème précis : permettre de scraper un nouveau site HTML statique en écrivant uniquement un fichier de configuration YAML, sans toucher au code Python. Le framework sépare une fois pour toutes les responsabilités (récupération, extraction, validation, stockage, orchestration) dans des composants réutilisables, et laisse à la configuration le soin de décrire ce qui est spécifique à un site donné : son URL, sa pagination, ses sélecteurs CSS, son backend de stockage.

### ***Quelles sont les limites de portée (ce qu'il ne fait pas intentionnellement) ?***

- Pas de rendu JavaScript : DataHarvest cible explicitement les sites HTML statiques (comme demandé dans le sujet). Un site nécessitant Selenium ou Playwright est hors périmètre.
- Pas de crawl distribué ni asynchrone : un seul processus, séquentiel, avec un délai configurable entre les requêtes. Adapté à un usage ponctuel ou à une petite veille périodique, pas à l'indexation de millions de pages.
- Pas d'authentification par formulaire : aucun mécanisme de login/session persistante au-delà des cookies gérés nativement par requests.Session().
- Pas de vérification automatique du robots.txt : contrairement à Scrapy, DataHarvest ne bloque pas lui-même les URL interdites. Cette vérification reste manuelle, à la charge du développeur avant d'écrire une configuration (voir section 4).

Ces exclusions sont assumées : le sujet demande un framework pédagogique généraliste sur du HTML statique, pas un concurrent de Scrapy.

### ***Quel site avez-vous choisi pour tester ? Pourquoi ?***

Le framework a été validé sur 5 sites couvrant 3 niveaux de difficulté différents (contrainte de diversité du sujet) : books.toscrape.com et quotes.toscrape.com (niveau 1, sites d'entraînement stables), fr.wikipedia.org et data.gouv.fr (niveau 2, sites publics réels avec des structures HTML plus irrégulières), et blogdumoderateur.com (niveau 3, site d'actualités avec pagination). books.toscrape.com sert de fil rouge dans ce rapport car c'est le cas le plus simple à suivre de bout en bout ; les quatre autres illustrent chacun une limite différente du modèle initial du framework, ce qui a motivé plusieurs évolutions d'architecture décrites en section 5.

## 2. Architecture et choix de conception

### ***Pourquoi une architecture en composants découplés plutôt qu'un script monolithique ? Quels sont les gains concrets ?***

Un script monolithique mélange HTTP, parsing et stockage dans le même flot d'exécution : modifier le format de sortie oblige à retoucher le code qui fait aussi les requêtes réseau, avec le risque d'introduire un bug ailleurs. En séparant Fetcher, Pipeline, Validator et Store, chaque composant a une responsabilité unique et une interface stable (une seule méthode publique principale par composant : `fetch()`, `process()`, `validate()`, `save()`).

Cette séparation a un coût initial (plus de fichiers, plus d'abstractions à comprendre avant de commencer à coder), mais ce coût est amorti dès le deuxième site scrapé. Le premier site (`books.toscrape.com`) a nécessité d'écrire les 7 composants ; chacun des quatre sites suivants n'a nécessité qu'un nouveau fichier de configuration YAML, sans toucher au code Python — exactement l'objectif du sujet.

Trois gains concrets observés pendant le développement :

- **Testabilité** : Pipeline et Validator ne font aucune requête réseau — les 59 tests unitaires du projet s'exécutent en moins de 2 secondes, sans dépendre de la disponibilité d'un site externe. Seul le test d'intégration (marqué `@pytest.mark.integration`) nécessite Internet. Sur un script monolithique, tester l'extraction seule aurait nécessité soit de vraies requêtes HTTP à chaque test (lent, fragile, dépendant du réseau), soit un mock enchevêtré dans toute la fonction.
- **Extensibilité** : ajouter le mode « par ligne » à `GenericPipeline` (voir plus bas) ou le pattern « liste → détail » (`dataharvest/detail.py`) n'a nécessité de modifier ni Fetcher, ni Validator, ni Store — la modification est restée localisée aux deux composants concernés (Config pour lire la nouvelle section YAML, Pipeline/Orchestrator pour l'exploiter). Un script monolithique aurait probablement mélangé cette logique d'enrichissement avec la boucle de pagination existante, augmentant le risque de régression sur le comportement déjà validé.
- **Lisibilité** : chaque fichier tient dans une seule responsabilité, et le nom de la classe suffit à deviner son rôle (Fetcher fetch, Validator valide, Store stocke). Un nouveau contributeur au projet (ou le binôme lui-même en reprenant le code après une pause) peut se concentrer sur un seul fichier de 60 à 100 lignes plutôt que sur un script de plusieurs centaines de lignes où tout est mêlé.

### ***Pourquoi BasePipeline est-elle une classe abstraite (ABC) et pas une classe normale avec des méthodes vides ?***

Une classe normale avec des méthodes vides (par exemple retournant `None` ou `pass`) permettrait d'instancier `BasePipeline` directement, ou d'oublier d'implémenter `process()` sans que rien ne le signale avant l'exécution — l'erreur n'apparaîtrait qu'au moment où le code appelle une méthode absente, potentiellement en production, après plusieurs pages déjà traitées.

Avec ABC et `@abstractmethod`, Python refuse d'instancier toute sous-classe qui n'implémente pas `process()` et `next_page_url()` : l'erreur remonte dès la création de l'objet (`TypeError: Can't instantiate abstract class`), pas au premier appel. C'est une garantie donnée à l'utilisateur du framework qui écrirait sa propre Pipeline personnalisée (par exemple pour un site à la structure trop irrégulière pour `GenericPipeline`, comme envisagé pour `books.toscrape.com` si l'on voulait extraire la catégorie depuis l'URL plutôt que depuis un

sélecteur CSS) : le contrat d'interface est vérifié par le langage lui-même, pas seulement par une convention de nommage, un commentaire, ou une relecture de code qui pourrait passer à côté de l'oubli.

Ce même raisonnement s'applique à BaseMiddleware : sans ABC, rien n'empêcherait un middleware personnalisé de n'implémenter que `process_request()` en oubliant `process_response()`, cassant silencieusement la chaîne au moment de la réponse plutôt qu'à l'écriture du code.

### ***Pourquoi le pattern middleware pour le Fetcher ? Comment auriez-vous géré le retry sans ce pattern ?***

Sans middleware, la logique de retry (backoff exponentiel, gestion du code 429, respect de l'en-tête `Retry-After`) serait codée en dur à l'intérieur de `Fetcher.fetch()`, mélangée à la logique HTTP elle-même. Ajouter un comportement supplémentaire — par exemple `RateLimitMiddleware`, qui garantit un délai minimum par domaine — obligerait à modifier `Fetcher` à chaque nouvel ajout, et à re-tester l'ensemble de la méthode `fetch()` à chaque fois plutôt que le seul middleware concerné.

Avec le pattern middleware (`BaseMiddleware`, une liste ordonnée traversée à chaque requête et chaque réponse), `Fetcher` reste inchangé quel que soit le nombre de middlewares actifs : il se contente d'itérer sur `self.middlewares` sans connaître leur contenu. C'est exactement le même principe que les middlewares Django/Flask : `Fetcher` ne connaît que l'interface `process_request/process_response`, jamais les détails de `LoggingMiddleware` ou `RetryMiddleware`.

Sans ce pattern, la seule alternative aurait été un empilement de `if/else` ou de flags booléens dans `fetch()` (`if self.logging_enabled: ...`, `if self.retry_enabled: ...`), rapidement illisible dès qu'on ajoute une troisième ou quatrième préoccupation transverse, et difficile à tester isolément : les tests de `RetryMiddleware` (backoff exponentiel, arrêt après le nombre maximal de tentatives) sont aujourd'hui indépendants de tout appel HTTP réel, ce qui n'aurait pas été possible avec une logique fusionnée dans `Fetcher`.

### ***Pourquoi le chargement de configuration via YAML plutôt que des arguments CLI ? Quand préféreriez-vous l'un ou l'autre ?***

Un fichier YAML décrit une structure de données arborescente (sélecteurs multiples, pagination, sections optionnelles comme `detail` ou `validator`) qu'il serait pénible d'exprimer avec des arguments `--flag` en ligne de commande — on se retrouverait avec des dizaines de flags répétés à chaque lancement, ou avec un format ad hoc pour encoder des dictionnaires dans une chaîne de caractères passée en argument (`--selectors '{"titre": "h2 a"}'`), fragile et illisible.

Le YAML permet aussi de versionner une configuration dans Git comme un artefact à part entière : `configs/wikipedia_pays.yaml` documente à lui seul tous les choix faits pour ce site, avec des commentaires expliquant chaque piège rencontré (liens de drapeaux, notes de bas de page, noms alternatifs) — une vraie mémoire du projet consultable des mois plus tard, ce qu'un historique de commandes shell ne permettrait pas facilement.

Les arguments CLI restent préférables pour des paramètres qui varient à chaque exécution et non par site : `--dry-run`, ou le chemin du fichier de sortie pour un export ponctuel (`python -m dataharvest export --from ... --to ...`). C'est le choix fait dans `app.py` : la configuration structurelle et stable dans le temps (sélecteurs, pagination, backend de stockage) va en YAML, les options d'exécution ponctuelles et changeantes vont en argument CLI. Un projet

qui n'aurait qu'un seul site fixe et jamais de variation de sélecteurs pourrait légitimement préférer tout en CLI ; DataHarvest, pensé pour couvrir un minimum de 5 sites différents, a fait le choix inverse.

***Pourquoi l'injection de dépendances dans le constructeur plutôt que des imports directs ?  
Illustrez avec un exemple concret de test.***

Si Orchestrator créait lui-même un vrai Fetcher qui fait de vraies requêtes HTTP (import direct, instanciation interne sans possibilité de substitution), tester Orchestrator.run() nécessiterait une connexion réseau réelle à chaque exécution des tests — lent, fragile face à l'indisponibilité temporaire d'un site, et incompatible avec une exécution en CI sans accès Internet. En injectant Fetcher (et les autres composants) via le constructeur, un test peut remplacer requests.Session.get par un mock sans jamais toucher le code d'Orchestrator :

```
with patch.object(orchestrator.fetcher.session, "get", side_effect=fake_get):    rapport =  
orchestrator.run()
```

C'est exactement le mécanisme utilisé dans tests/test\_orchestrator.py::test\_orchestrator\_detail\_pattern\_enriches\_items : la page liste et la page de détail sont simulées par un simple générateur Python qui retourne des chaînes HTML statiques préparées à l'avance, ce qui permet de vérifier que le pattern « liste → détail » fusionne bien capitale et continent dans l'item final, en quelques millisecondes et sans dépendre de la disponibilité de Wikipédia au moment du test. Le même principe permet de tester Orchestrator avec un Store factice (backend json pointant vers un fichier temporaire pytest tmp\_path) plutôt que d'écrire réellement dans output/, évitant toute pollution entre exécutions de tests successives.

### 3. Comparaison avec Scrapy

#### 5 fonctionnalités que Scrapy offre et que DataHarvest n'offre pas

| Fonctionnalité Scrapy                                                            | Pourquoi absente de DataHarvest                                                                                                                                                                                                                    |
|----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Moteur asynchrone (Twisted)                                                      | Hors scope : réécrire Fetcher/Orchestrator autour d'une boucle d'événements aurait démultiplié la complexité pour un projet de 5h, et cassé la lisibilité pédagogique recherchée.                                                                  |
| Respect automatique de robots.txt (ROBOTSTXT_OBEY)                               | Non implémenté par manque de temps ; la vérification reste manuelle (voir section 4). Une vraie lacune à combler avant tout usage en production.                                                                                                   |
| scrapy shell (test interactif de sélecteurs)                                     | Complexité : nécessiterait un mode interactif REPL branché sur Fetcher + BeautifulSoup, hors périmètre du sujet qui demande un CLI simple (crawl/export/validate).                                                                                 |
| Déduplication de requêtes à l'échelle du crawl (dupefilter)                      | DataHarvest ne déduplique qu'au moment du stockage (Store, contrainte UNIQUE sur l'URL), pas au moment de la requête : une même URL peut être fetchée deux fois si elle apparaît sur deux pages liste différentes. Hors scope par manque de temps. |
| Écosystème de stats/extensions (AutoThrottle adaptatif, telnet console, signals) | Hors scope : DataHarvest a un middleware fixe (Logging, Retry, RateLimit en option) mais pas de système de plugins/signaux généralisé.                                                                                                             |

#### Dans quels cas un développeur préférerait-il DataHarvest à Scrapy ? Deux scénarios concrets

**Scénario 1 — Prototypage pédagogique ou audit rapide d'un site :** un développeur qui découvre un nouveau site et veut vérifier en quelques minutes si ses données sont exploitables n'a pas besoin de scrapy startproject, d'un dossier spiders/, ni de comprendre le cycle de vie Twisted. Une configuration YAML de 15 lignes suffit, et python -m dataharvest crawl --dry-run donne un retour immédiat. C'est le scénario vécu pendant ce projet : chaque nouveau site (Wikipédia, data.gouv.fr) a été validé en quelques itérations rapides sur des extraits HTML avant tout crawl réel.

**Scénario 2 — Script ponctuel ou tâche cron à faible volume :** pour une veille quotidienne de quelques dizaines de pages (par exemple surveiller les nouveaux jeux de données d'une organisation sur data.gouv.fr), la simplicité de déploiement (un seul processus Python synchrone, sans réacteur Twisted à superviser) l'emporte sur le gain de performance de Scrapy, qui ne se manifeste vraiment qu'à partir de volumes bien plus importants.

#### Impact synchrone vs asynchrone : calcul théorique pour 100 pages avec $DOWNLOAD\_DELAY=1s$

Hypothèses : temps de réponse serveur négligé au premier ordre par rapport au délai de politesse (~0,2 s de réponse contre 1 s de délai), délai imposé de 1 s entre deux requêtes vers le même domaine.

**DataHarvest (séquentiel) :**  $T \approx N \times (\text{délai} + \text{temps de réponse}) = 100 \times (1 + 0,2) \approx 120$  secondes (2 minutes), puisque chaque requête attend la précédente.

**Scrapy (concurrent, ex. `CONCURRENT_REQUESTS_PER_DOMAIN = 8`) :** en théorie, les 8 requêtes en vol se partagent le même délai de politesse par domaine ; le temps total se rapproche de  $T \approx (N / C) \times (\text{délai} + \text{temps de réponse}) = (100 / 8) \times 1,2 \approx 15$  secondes.

Gain théorique de l'ordre de 8×, proportionnel au nombre de connexions concurrentes autorisées. Ce calcul reste théorique : la contrainte réelle est souvent le serveur cible (throttling, blocage anti-bot) plutôt que le client, et nous avons déjà observé dans un TP précédent de ce module qu'un gain attendu (mode headless Selenium) peut être bien inférieur

à la théorie une fois mesuré en conditions réelles — un rappel que ce calcul mérite d'être vérifié empiriquement plutôt que pris pour argent comptant.

## 4. Cadre légal et éthique

### *Analyse du robots.txt du site scrapé*

books.toscrape.com est un site explicitement créé par Scrapinghub (aujourd'hui Zyte) comme terrain d'entraînement public au web scraping, sans finalité commerciale ni donnée sensible. La vérification de <https://books.toscrape.com/robots.txt> fait partie du protocole suivi avant d'écrire chaque configuration de ce projet.

[À COMPLETER PAR LE BINÔME : coller ici le contenu exact obtenu en visitant chaque robots.txt des 5 sites (books.toscrape.com, quotes.toscrape.com, fr.wikipedia.org, data.gouv.fr, blogdumoderateur.com), et confirmer explicitement qu'aucune URL scrapée par nos configurations ne figure dans une règle Disallow.]

Point important à assumer honnêtement : DataHarvest ne vérifie pas automatiquement le robots.txt au moment de l'exécution (contrairement à Scrapy et son middleware ROBOTSTXT\_OBEY). Cette vérification a été faite manuellement, site par site, avant d'écrire chaque fichier de configuration — ce qui fonctionne pour 5 sites choisis avec soin, mais ne passerait pas à l'échelle sans discipline supplémentaire. C'est identifié comme une limite du framework, pas comme un choix éthique délibéré (voir aussi section 6, perspectives).

### *Quelle base légale (RGPD Art. 6) justifie la collecte de données ?*

Les cinq sites scrapés diffusent des informations à but explicitement public et réutilisable : catalogue de livres et citations d'entraînement (books.toscrape.com, quotes.toscrape.com), contenu encyclopédique sous licence libre CC BY-SA (Wikipédia), catalogue de données publiques ouvertes par construction légale (data.gouv.fr, licence ouverte Etalab), et articles de presse librement accessibles (Le Blog du Modérateur). Le fondement retenu est l'article 6.1.f du RGPD (intérêt légitime) : la finalité pédagogique du projet (apprentissage du web scraping) ne nécessite le consentement d'aucune personne physique, puisqu'aucune des données extraites ne concerne un individu identifié à titre privé (voir point suivant).

### *Les données collectées sont-elles des données personnelles ?*

Non, pour les cinq configurations retenues :

- books.toscrape.com / quotes.toscrape.com : titres de livres fictifs, citations d'auteurs et de personnages publics déjà largement diffusées, prix et notes — aucune personne physique réelle concernée par une collecte de données la concernant.
- Wikipédia : noms de pays, capitales, continents — des données géographiques et institutionnelles, pas des données à caractère personnel.
- data.gouv.fr : titres de jeux de données et noms d'organisations (personnes morales) — pas de données personnelles.
- blogdumoderateur.com : titres d'articles, dates, catégories — contenu éditorial public, sans donnée personnelle extraite (les noms de journalistes ne font pas partie des champs scrapés).

Si un futur usage de DataHarvest venait à cibler des données à caractère personnel (profils, avis nominatifs, coordonnées), une analyse RGPD spécifique serait nécessaire avant toute collecte — ce n'est pas le cas ici.

### *Mécanismes techniques mis en place pour être un « bon citoyen » du web*

| Mécanisme | Où dans le code |
|-----------|-----------------|
|-----------|-----------------|



|                                                                     |                                                                                                        |
|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| User-Agent identifiable                                             | config.fetcher.user_agent, envoyé sur chaque requête via requests.Session().headers                    |
| Throttling configurable                                             | config.fetcher.delay, respecté entre chaque page ET entre chaque page de détail (pattern liste→détail) |
| Retry avec backoff exponentiel (pas de martèlement en cas d'erreur) | RetryMiddleware.backoff_delay() : $\text{délai} \times 2^{\text{tentative}}$                           |
| Respect du code 429 et de l'en-tête Retry-After                     | Fetcher.fetch()                                                                                        |
| Déduplication (UNIQUE url)                                          | Store, contrainte SQL UNIQUE + INSERT OR IGNORE (sqlite), ou vérification manuelle (json/csv)          |
| Timeout configurable (pas de connexion qui bloque indéfiniment)     | config.fetcher.timeout                                                                                 |

## 5. Difficultés et rétrospective

### *Quelle a été la difficulté technique principale ? Comment l'avez-vous résolue ?*

La difficulté la plus significative a été le modèle d'alignement « à plat » de GenericPipeline sur la page Wikipédia des pays. Le modèle initial sélectionne chaque champ sur toute la page puis reconstruit les items en associant les résultats par position (index *i* du premier champ ↔ index *i* des autres champs). Ce modèle suppose implicitement qu'un champ apparaît exactement une fois par item.

Sur Wikipédia, une même cellule « forme courte » contient parfois plusieurs liens pour un seul pays (nom courant, translittération alternative, nom historique — par exemple Afghanistan / Afghānistān / Émirat islamique d'Afghanistan dans la même cellule). Plusieurs correctifs successifs par sélecteur CSS (:not()) pour exclure les liens d'image ou de note de bas de page, :first-of-type, :first-child) se sont révélés insuffisants ou fragiles face à des imbrications HTML variables.

La résolution définitive a nécessité une évolution d'architecture plutôt qu'un simple ajustement de sélecteur : l'ajout d'un mode « par ligne » à GenericPipeline (row\_selector dans la configuration), qui délimite d'abord chaque conteneur de ligne puis cherche chaque champ à l'intérieur de ce seul conteneur via select\_one(). Ce mode garantit qu'un seul résultat est retenu par ligne, quelle que soit la profondeur d'imbrication des éléments alternatifs — une garantie qu'aucune combinaison de pseudo-classes CSS ne pouvait offrir de façon fiable.

Un second bug notable, plus subtil, a été rencontré sur quotes.toscrape.com : le sélecteur div.tags a.tag:first-child ne retournait jamais de résultat, car le premier enfant réel de div.tags était une balise <meta class="keywords"> insérée avant les liens de tags — :first-child exige la toute première position parmi tous les types d'éléments, alors que :first-of-type ne compte que parmi les éléments de même balise. La différence entre ces deux pseudo-classes, généralement anodine, devient déterminante face à du HTML réel imprévisible.

Un troisième problème, de nature différente, est apparu sur books.toscrape.com : les prix s'affichaient « Â£51.77 » au lieu de « £51.77 ». La cause n'était pas un sélecteur mais l'encodage HTTP : requests utilise ISO-8859-1 par défaut lorsque le serveur ne déclare pas explicitement de charset dans l'en-tête Content-Type, ce qui corrompt les caractères UTF-8 multi-octets comme le symbole livre sterling. La correction — forcer response.encoding = response.apparent\_encoding avant de lire response.text dans Fetcher.fetch() — a été appliquée une seule fois au niveau du framework, bénéficiant immédiatement aux 5 configurations sans modification supplémentaire, illustration concrète du gain apporté par la centralisation de la logique HTTP dans un composant unique plutôt que dupliquée site par site.

### *Qu'est-ce que vous referiez différemment si vous recommenciez ?*

- Concevoir le mode « par ligne » de GenericPipeline dès la première version plutôt qu'en réaction à un bug découvert tardivement sur Wikipédia : le modèle « à plat » aurait dû être traité comme un cas particulier optimisé, pas comme le modèle par défaut.
- Vérifier systématiquement le HTML réel d'un site (via un extrait collé depuis l'inspecteur navigateur) avant d'écrire le premier sélecteur, plutôt que de partir d'hypothèses construites sur des exemples similaires déjà rencontrés — plusieurs itérations auraient été évitées sur Wikipédia et data.gouv.fr.

- Ajouter robots.txt comme vérification automatique dans Fetcher dès le départ, plutôt que de la laisser en vérification manuelle externe au framework.

### ***Quelle fonctionnalité n'avez-vous pas eu le temps d'implémenter ? Comment l'implémenteriez-vous ?***

Le RateLimitMiddleware existe dans middleware.py (garantit un délai minimum par domaine) mais n'est pas branché par défaut dans Orchestrator, qui n'active que LoggingMiddleware et RetryMiddleware. Une extension naturelle serait de le rendre configurable depuis le YAML (par exemple un champ `fetcher.min_delay_per_domain`), pour permettre de scraper plusieurs domaines en parallèle à l'avenir sans risquer de surcharger l'un d'eux — utile si le framework évoluait un jour vers un mode multi-sites simultané (voir aussi section 6).

### ***Répartition des tâches : qui a fait quoi ? Est-ce que cela se voit dans Git ?***

La répartition a suivi le planning suggéré par l'énoncé du projet (section 9, « Planning suggéré »), qui propose une répartition par sprint entre Étudiant A et Étudiant B plutôt qu'une répartition libre. Ce choix a été délibéré plutôt que subi : négocier une répartition ad hoc en début de projet aurait consommé un temps précieux sur un créneau de 7h, alors que le découpage proposé par le sujet suit déjà l'ordre naturel des dépendances entre composants (Config avant Pipeline, Middleware avant que Fetcher n'en ait besoin pour le retry, Validator et Store indépendants et donc parallélisables). Suivre ce planning a permis de démarrer le travail en parallèle dès le sprint 1 sans phase de conception d'équipe supplémentaire.

**Tahina :** Config, Pipeline (dont les deux extensions ajoutées en cours de projet : le mode « par ligne » et le pattern « liste → détail »), Orchestrator, la suite de tests associée, le README, et la configuration + le débogage des 5 sites ([books.toscrape.com](http://books.toscrape.com), [quotes.toscrape.com](http://quotes.toscrape.com), [blogdumoderateur.com](http://blogdumoderateur.com), Wikipédia, [data.gouv.fr](http://data.gouv.fr)).

**Houda :** Middleware (Logging, Retry, RateLimit), Fetcher, Validator, Store, App CLI (`crawl/export/validate`), en suivant fidèlement l'interface attendue par Orchestrator sans modification nécessaire une fois intégrés.

Cette répartition se retrouve dans l'historique Git : les commits touchant `dataharvest/config.py`, `dataharvest/pipeline.py`, `dataharvest/orchestrator.py` et `configs/*.yaml` sont signés Tahina, tandis que ceux touchant `dataharvest/middleware.py`, `dataharvest/fetcher.py`, `dataharvest/validator.py`, `dataharvest/store.py` et `dataharvest/app.py` sont signés Houda. [À COMPLETER : vérifier avec `git log --oneline --author="..."` pour chaque membre, et citer ici le nombre exact de commits de chacun ainsi que le nombre total du dépôt.]

## 6. Perspectives

### ***Proposition d'une 6<sup>e</sup> brique : Monitor***

Une brique Monitor viendrait s'ajouter aux 5 existantes pour surveiller la santé d'un crawl dans la durée (utile pour une veille périodique automatisée). Elle recevrait le rapport de session produit par `Orchestrator.run()` à chaque exécution, comparerait les métriques au fil du temps, et déclencherait une alerte en cas d'anomalie (par exemple un taux de rejet anormalement élevé, signe probable qu'un sélecteur a cessé de fonctionner suite à un changement du site).

```
class Monitor:
    def __init__(self, seuil_taux_rejet: float = 0.3, historique_path: str = "monitor.json"):
        self.seuil_taux_rejet = seuil_taux_rejet
        self.historique_path = historique_path
    def enregistrer(self, rapport: dict) -> None:
        """Ajoute le rapport de session à l'historique persistant."""
    def detecter_anomalie(self, rapport: dict) -> str | None:
        """Retourne un message d'alerte si le taux de rejet dépasse le seuil, ou si items_trouves chute brutalement par rapport à la moyenne historique (site probablement modifié / selecteurs cassés)."""
    def alerter(self, message: str) -> None:
        """Point d'extension : email, webhook Slack, log fichier..."""
```

`Orchestrator.run()` appellerait `Monitor.enregistrer(rapport)` puis `Monitor.detecter_anomalie(rapport)` en toute fin d'exécution, sans dépendance dans l'autre sens (Monitor ne connaît qu'un dict de rapport, jamais Orchestrator lui-même) — cohérent avec le principe d'injection de dépendances déjà appliqué au reste du framework.

### ***Comment rendre DataHarvest compatible avec les sites JavaScript sans dépendre de Selenium ?***

Playwright propose un mode de pré-rendu headless plus rapide et plus stable que Selenium (API asynchrone native, meilleure gestion des attentes réseau). Le point d'intégration le plus propre serait au niveau de `Fetcher` : ajouter un mode `config.fetcher.engine: playwright` qui, au lieu d'utiliser `requests.Session().get()`, lancerait une page Playwright headless, attendrait un sélecteur de chargement (équivalent du `WebDriverWait` déjà pratiqué dans les TP précédents), puis retournerait `page.content()` à `Pipeline` exactement comme le ferait `response.text` aujourd'hui. Aucune modification ne serait nécessaire dans `Pipeline`, `Validator`, `Store` ou `Orchestrator`, puisque tous consomment déjà du HTML brut sous forme de chaîne de caractères, indépendamment de son origine — bénéfice direct de l'architecture en composants découplés.

Une alternative plus légère serait un service de pré-rendu externe (type `Splash` ou un simple service Playwright exposé en HTTP), appelé par `Fetcher` comme un proxy : le reste du framework resterait alors inchangé.

### ***Que faudrait-il ajouter pour distribuer DataHarvest sur PyPI ?***

- Un fichier `pyproject.toml` (ou `setup.py`) déclarant le nom du paquet, la version, les dépendances (`requests`, `beautifulsoup4`, `lxml`, `pyyaml`) et le point d'entrée CLI (`console_scripts` pointant vers `dataharvest.app:main`, pour que la commande `dataharvest` fonctionne sans `python -m`).
- Un nom de paquet vérifié disponible sur PyPI, et un README suffisamment complet pour servir de page de présentation du paquet (déjà largement couvert par le `README.md` existant).

- Un versionnage sémantique explicite (le fichier `__init__.py` contient déjà `__version__`), et une action CI (GitHub Actions) qui publie automatiquement sur tag Git via twine upload.
- Retirer ou neutraliser les chemins codés en dur (ex: `output/`) pour que l'installation en tant que dépendance externe n'écrive pas par défaut dans le répertoire courant d'un projet tiers sans configuration explicite.

## Bibliographie / Sources

- Documentation officielle Scrapy — <https://docs.scrapy.org/>
- Documentation Beautiful Soup 4 — <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>
- Documentation Requests — <https://requests.readthedocs.io/>
- books.toscrape.com et quotes.toscrape.com — sites d'entraînement publics maintenus par Zyte (ex-Scrapinghub)
- Documentation Playwright (pour la section Perspectives) — <https://playwright.dev/python/>